

Fully Memristive Spiking Neural Network with Forward-Forward Learning

A dissertation submitted in partial fulfilment of

the requirements for the degree of

BACHELOR OF SCIENCE Computer in Science

in

The Queen's University of Belfast

by

OOI HONG YAN (PETER)

40374182

July 20, 2026

SCHOOL OF ELECTRONICS, ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

CSC3002 – COMPUTER SCIENCE PROJECT

Dissertation Cover Sheet

A signed and completed cover sheet must accompany the submission of the Computer Science dissertation submitted for assessment. Work submitted without a cover sheet will NOT be marked.

Student Name:	OOI HONG YAN (PETER)
Student Number:	40374182
Project Title:	Fully Memristive Spiking Neural Network with Forward-Forward Learning
Supervisor:	XINMING SHI

Declaration of Academic Integrity

Before submitting your dissertation please check that the submission:

1. Has a full bibliography attached laid out according to the guidelines specified in the Student Project Handbook
2. Contains full acknowledgement of all secondary sources used (paper-based and electronic)
3. Contains full acknowledgement of generative AI usage throughout the project
4. Does not exceed the specified page limit
5. Is clearly presented and proof-read
6. Is submitted on, or before, the specified or agreed due date. Late submissions will only be accepted in exceptional circumstances or where a deferment has been granted in advance.

By submitting your dissertation you declare that you have completed the tutorial on plagiarism at <http://www.qub.ac.uk/cite2write/introduction5.html> and are aware that it is an academic offence to plagiarise. You declare that the submission is your own original work. No part of it has been submitted for any other assignment and you have acknowledged all written and electronic sources used.

If selected as an exemplar, I agree to allow my dissertation to be used as a sample for future students.

Student's signature: OOI HONG YAN (PETER)

Date of submission: July 20, 2026

Acknowledgements

I would like to express my sincere gratitude to my supervisor, ****Xinming Shi****, for their invaluable guidance, patience, and support throughout the duration of this project. Their insights into spiking neural networks and the Forward-Forward learning algorithm were instrumental in the completion of this dissertation.

I would also like to thank the School of Electronics, Electrical Engineering and Computer Science for providing the academic framework and curriculum for this project.

Finally, I am grateful to my family and friends for their continuous encouragement during the development of this project.

Abstract

We propose a fully memristive spiking neural network (MSNN) trained with the Forward-Forward (FF) learning algorithm to enable energy-efficient and scalable neuromorphic computing. In the proposed architecture, memristive integrate-and-fire (MIF) neurons and synapses are jointly realized using analog memristor devices, forming compute-in-memory modules with strictly local learning signals. By replacing global error backpropagation with layer-wise FF updates, the network avoids large memory-intensive computational graphs and relies solely on local learning signals that are more consistent with biologically inspired neural processing. This design supports fully analog operation without digital interfacing components such as ADCs or centralized control logic. Experimental evaluation on MNIST, KMNIST and FMNIST demonstrate competitive performance, achieving up to 97.58%, 87.95% and 89.23% classification accuracies respectively. These results indicate that fully memristive SNNs with FF learning provide a promising pathway toward low-power, scalable neuromorphic systems suitable for future edge and in-memory AI applications.

Contents

1	Introduction	6
1.1	The Great Wall: Energy and Memory Constraints in Modern Silicon	6
1.2	Spiking Neural Networks	7
1.3	Forward Forward	8
2	Contributions	9
3	Design	9
3.1	Fully Memristive Spiking Neural Networks	9
3.2	The Forward-Forward Paradigm	10
3.3	FF in Spiking Neural Networks	10
3.4	Memristive Integrate-and-Fire Neuron Dynamics	11
3.5	Forward-Forward Learning with Temporal Memristive Dynamics	13
4	Implementation	15
4.1	Technical Details	15
4.2	Optimizations	15
4.3	MIF module	15
4.4	System Configuration and Layer Hierarchy	16
4.5	Dataset	17
4.6	Training and Inference	17
5	Testing	19
5.1	Training Loss and Data Integrity	19
5.2	MIF Single Neuron Simulation	19
5.3	Explosion Test	19
6	Experiment	21
6.1	Result	21
6.2	Accuracy Comparison	21
6.3	Energy and Area Efficiency	22
7	Conclusion	23
7.1	Evaluation	23
7.2	Future Work	24
7.3	Limitations	24
7.4	Implications	25
	References	26

A Appendices **29**

A.1 AI Acknowledgements 29

1 Introduction

1.1 The Great Wall: Energy and Memory Constraints in Modern Silicon

The rapid ascension of neural networks has undoubtedly revolutionized the computing landscape, where cognitive abilities once exclusive to humanity can now be simulated on performant silicon. In the early days, artificial intelligence heavily relied on General-Purpose CPUs. However, as the nature of the hardware was inherently sequential, they exhibited slow and impractical performance. With the realization of the highly parallelizable Backpropagation, models eventually gravitated toward the superior performance offered by Graphics Processing Units (GPUs), whose massively parallel architectures better suit modern neural networks. Today, major corporations deploy Large Language Models (LLMs) such as ChatGPT and Gemini using massive supercomputer clusters across a multitude of datacenters. These installations contain thousands of GPUs, incurring millions in daily operational costs and consuming enough energy to power a small city.

Furthermore, as AI expands its domain beyond data centers into edge computing and ultra-low-power environments, these power limitations have become increasingly prohibitive. Despite their parallelization strengths, GPUs remain bound to architectures that are mandatorily optimized for graphical rendering. While specialized hardware developments such as Tensor Cores have provided significant performance gains, the fundamental "Memory Wall" and the inefficiencies of clocked processing remain unaddressed. Consequently, an abundance of neural-inspired optimizations remain unreachable with traditional silicon systems, driving the search for neuromorphic alternatives.

Apart from scaling limits, the current hardware critically suffers from three crucial systemic failures. To begin, energy inefficiency. Current accelerators consume orders of magnitude more power than their biological counterparts, rendering them impractical for autonomous edge devices and creating unsustainable energy sinks for global supercomputing hubs. Current models also heavily rely on Biological Abstraction, reducing neurons to purely mathematical functions to facilitate high-speed parallelization, but potentially sacrifices nuances like temporal encoding which might be essential to biological efficiency. This abstraction is also bound by the limitations brought by the Von Neumann Bottleneck, the "memory wall" from the data transfers between VRAM and processing .

Finally, the reliance on Global Backpropagation presents a significant architectural and biological hurdle. Due to the fact that this methodology requires the retention of massive computational graphs in memory, this restricts the already scarce memory. Given that neurobiological evidence suggests the brain operates without such global gradients, there is no doubt that current training paradigms have hit the wall of diminishing returns. To bypass these limitations, adoption towards neuromorphic architectures which prioritize local learning rules and in-situ memory has now evolved into an inevitable mechanical necessity.

To address these systemic failures, significant advancements have been made in neuromorphic research, with chip manufacturers increasingly experimenting with Neural Processing Units (NPU) to allow for the direct simulation of neurons at the hardware level. However, hardware alone is insufficient, as now the model must also be written to be asynchronous in nature. This requires new complexities beyond the traditional static weight abstraction of the neuron to the sophisticated temporal dynamics of Spiking Neural Networks (SNNs).

By mimicking the biological brain’s ability to process information only when specific “events” or spikes occur, extreme sparsity can be achieved. An asynchronous neuromorphic approach only requires the activation of specific circuitry needed to process a spike, effectively bypassing the energy sinks and idling costs that plague current accelerators. Furthermore, the precise timing of when a neuron fires has the potential to encode higher levels of information density without the need for massive, VRAM-heavy parameter counts. This shift towards event-driven, in-situ processing represents a fundamental departure from traditional silicon logic, offering a path to finally break through the memory wall.

1.2 Spiking Neural Networks

Due to the systemic limitations of current hardware, alternative computational paradigms have moved to the forefront of architectural research. Extensive neuroscientific study serves as the primary biological blueprint for this shift. The human brain remains an engineering marvel, sustaining high-level cognitive processing, which can still surpasses the capabilities of cutting-edge AI in most real-world scenarios on approximately 20 Watts. This highlights a potential efficiency gap of several orders of magnitude in comparison to the megawatt-scale consumption of modern GPU clusters.

This disparity is largely attributed to the brain’s reliance on discrete, asynchronous spikes rather than the continuous, high-precision numerical activations used in traditional artificial neural networks. Because traditional numerical computations are typically synchronous, every neuron in a given layer is updated at every timestep, regardless of whether the input data has changed or is even relevant to the current task. This “always-on” approach results in significant power wastage, as energy is consumed for redundant calculations that do not contribute to the final output. In contrast, the spiking paradigm allows for an event-driven execution where energy is consumed only upon the occurrence of a significant signal, mirroring the sparse efficiency of biological systems.

To capture this biological efficiency, Spiking Neural Networks (SNNs) impose neuroscience-inspired constraints on modern deep learning algorithms. The core of this transition involves moving away from “full-precision” activations, where every signal is a complex, power-hungry float, and towards temporally encoded representations. Within SNNs, information is carried by the timing of the spikes themselves. When implemented on specialized neuromorphic hardware, this shift allows for a substantial reduction in both energy consumption and inference latency [2, 3].

Utilizing the approach of discrete temporal events, the hardware avoids the overhead of traditional synchronous architectures. These advantages have positioned SNNs as a promising candidate for practical, brain-inspired computing. It has been demonstrated that low inference latency and substantial efficiency improvements are significant hallmarks of this methodology. This is especially relevant in the realm of “edge” devices, such as wearable sensors, cameras and autonomous drones, as the ability to process complex data with minimal power makes SNNs a promising and prominent field to provide the architectural foundation for intelligence that is truly “everywhere”, operating within the strict energy constraints of the real world.

Although SNNs promises drastically higher efficiencies in suitable hardware, implementations into those hardware and the training of such networks remains significant mathematical barriers. Because spikes are discrete, thresholding events, they are fundamentally non-differentiable, thus standard calculus is largely in-

applicable as a learning function. Because of this Training SNNs has traditionally relied on surrogate gradient methods to address those discrete spike events. Surrogate gradient methods use mathematical approximations to fake the smooth curves needed for learning [4]. The emergence of memristive devices gives rise to a new approach. Unlike standard transistors, memristors are stateful and have resistances dependent upon its previous history, hence it has “memory”. By exploiting these intrinsic physics, memristive spiking neurons can generate continuous analog voltage trajectories. Since these physical signals are naturally smooth, they remain differentiable, eliminating the need for those mathematical approximations [5]. This capability enables the construction of fully memristive SNNs in where both synaptic and neuronal dynamics are realized within the physical substrate.

Currently however, existing training frameworks fail to exploit the device-level differentiability advantage and opted for global error backpropagation [6–8]. These approaches are in conflict with the fundamental architecture of the memristor. To put this into perspective, global backpropagation requires memory-intensive computational graphs, backward error propagation, and tightly coordinated weight updates across layers, whereas memristive crossbar architectures have, local state evolution, unidirectional dataflow, and device-level variability. Forcing a global backpropagation onto memristors requires massive energy overhead and complex “handshaking” between layers that cancels out the hardware’s natural efficiency and severely limits the practical scalability of fully memristive neuromorphic systems.

1.3 Forward Forward

The Forward–Forward (FF) learning paradigm [9] offers a natural alternative that aligns with both the physical constraints of memristive hardware and the principles of biological learning. This function replaces the global error backpropagation of traditional AI with “greedy” layer-wise learning, where each layer is optimized based on a local “goodness” score derived from the activity of its neurons. Instead of a backward pass, the system performs two forward passes, one with “positive” (real) data and one with “negative” (corrupted or generated) data. A local optimization can then be obtained with the loss function, allowing the layers to learn to distinguish between them by maximizing goodness for positive data and minimizing it for negative data. By replacing global error backpropagation with layer-wise local goodness optimization, FF enables training using forward-only signal propagation and inference-style operations that map directly onto memristive compute-in-memory substrates [10].

Since each layer is optimized independently based solely on local neuronal activity, in addition to the removal of backward error transport, it provides independence from weight symmetry (where forward and backward weights must match perfectly), or centralized control [11]. From a neuroscience perspective, this locality enhances biological plausibility, as synaptic updates depend on feedforward activity correlations rather than non-local error signals. Together, these properties make FF particularly well suited for fully memristive spiking neural networks, providing a learning mechanism that is energy-efficient, hardware-compatible, and consistent with brain-inspired computation.

Critically, this independent, layer-by-layer approach provides the robustness needed for real-world hardware. Because learning is local, the system is far more resilient to device-level variability. In a global system, one “noisy” memristor in the first layer can ruin the math for the entire network, whereas FF forces each

layer to adapt to the actual, “messy” physical behavior of the specific devices it contains.

2 Contributions

In this work, we develop a novel fully memristive spiking neural network (MSNN) trained using the Forward-Forward (FF) learning paradigm. This framework enables local, forward-only learning directly on analog memristive neuron and synapse models. By leveraging the intrinsic dynamics of Memristive Integrate-and-Fire (MIF) neurons, the proposed system eliminates the need for global backpropagation, surrogate gradients, or auxiliary digital control circuitry. Experimental evaluations demonstrate that FF-trained MSNNs can achieve competitive performance while maintaining the hardware compatibility and biological plausibility required for ubiquitous, brain-like AI.

Our main contributions are as follows:

- We integrate Forward-Forward learning into a biologically inspired fully memristive spiking neural network, enabling layer-wise local training using forward-only signal propagation.
- We demonstrate that the proposed framework achieves competitive accuracy on neuromorphic benchmarks, including MNIST, FMNIST, and KMNIST, outperforming existing memristive SNN models and achieving performance comparable to that of baseline software ANNs and software SNNs.
- We show that Forward-Forward learning provides an effective and hardware-compatible alternative to backpropagation for fully memristive SNNs, eliminating the need for ADC/DAC interfaces and backward signal paths, thereby enabling more energy-efficient and area-efficient fully analog neuromorphic systems.

3 Design

3.1 Fully Memristive Spiking Neural Networks

Currently, fully memristive SNNs implementations frequently suffer from physical discrepancy, as they are either optimized as idealized digital abstractions prior to being mapped into potentially non-ideal hardware [6, 7], or rely on heuristic rules like Spike-Timing-Dependent Plasticity (STDP), which despite its biological inspirations, lack a clear optimization objective for rigorous mathematical optimization [12]. Our method replaces these “black-box” neurons with a physically grounded Memristive Integrate-and-Fire (MIF) model.

Rather than utilizing simplified surrogate-based gradients to patch the non-differentiable nature of discrete spikes, our system is truly continuous. By modeling the temporal equations with real-world parameters voltage (V), memristive conductance ($G1, G2$), capacitance (C) and device resistance (R_{on}/R_{off}), we ensure a faithful manifestation of analog dynamics.

In modern works, existing simulators tend to treat memristors as static components or separate the learning phase from the device physics [13]. Our SPICE-level framework embeds the MIF neuron directly into a differentiable computational graph, providing the optimizer the visibility into the internal state variables

(x_1, x_2) . The framework tracks the entire voltage trajectory over time, enabling the application of local Backpropagation Through Time (BPTT) to calculate the gradients accordingly through the computational graph's differential equations. This allows for the optimization of the analog circuit's inherent physics as if it were a trainable dynamic component.

3.2 The Forward-Forward Paradigm

The Forward-Forward (FF) algorithm was initially introduced as a theoretical alternative to Backpropagation (BP) [9] and its applications have far expanded into specialized neuromorphic domains as a biologically plausible alternative. A major advantage of replacing the global backward pass with two layer-wise forward passes, one with positive and the other with negative data, is the resolution of the "weight transport problem" [11] and the unrealistic requirement for hardware to maintain perfectly symmetric weight knowledge during a backward pass.

While the original FF focused on rate-based networks, recent investigations have explored its efficacy across various architectures. For instance, Lorberbom [14] examined layer collaboration within the FF framework, highlighting the potential for decentralized learning. An issue of Hinton's initial rate-based implementation is apparent in its extreme layer wise isolation, leading to a severe lack of layer collaboration. When a layer purely optimizes only its own local objective, it fails to maintain subtle data features that subsequent layers require to learn more complex, hierarchical representations. Lorberbom's highlighted the requirement for a balance between local autonomy and global information flow. His work suggested the use of objective functions, where each layer's loss is conditioned on the performance of all preceding layers. By forcing the network to optimize for a "chain" of goodness rather than isolated snapshots, Lorberbom demonstrated that early layers could be incentivized to preserve important information the might be needed for future processing.

A more recent exploration from Gong et al. [15] proposes reshaping the FF objective to solve the inefficiency of Hinton's original framework, which requires multiple forward passes due to its dependence on one-hot label encoding. To do so, they created FAUST (Forward-forward Algorithm Unified with Similarity-based Tuplet loss), transferring the loss calculation from "goodness" to the similarity between images in the same class versus those in different classes. However, this approach requires simultaneous processing of anchor, positive, and negative samples, which imposes a heavy memory and energy burden on memristive systems. Our work adopts the original Hinton method by utilizing a temporal integration approach to tackle the limited-resource environment of analog hardware.

3.3 FF in Spiking Neural Networks

Adapting FF to the spiking domain is a rapidly expanding field. Taylor et al. [16] demonstrated that single-spike SNN training could be robustly accelerated, laying the groundwork for low-latency temporal tasks. By focusing solely on time-to-first-spike as the primary information carrier, they provided a vital proof-of-concept for the plausibility of FF in spiking architectures. Another prominent work was presented by Ororbia [17], introducing contrastive signal-dependent plasticity (CSDP) to link self-supervised learning directly

to spiking circuits. This approach encodes positive and negative data into distinct temporal signals, utilizing state persistence to drive synaptic updates. This behavior is highly compatible with memristive characteristics, as the conductance state is proportional to the cumulative history of input voltages.

The challenge of “gradient approximation” in SNNs has led to several gradient-free frameworks. Xu et al. [18] proposed the FFGAF-SNN framework, which bypasses surrogate gradients entirely by utilizing the FF local objective. This framework works by disregarding the complicated and costly surrogate gradient mathematics and simply treats the spiking activations themselves as the primary metric for output “goodness” of a layer rather than the derivative of each spike. Our work possesses major similarities to this work, leveraging low-cost calculations to allow our MIF model to update the memristive conductance (G_1, G_2) based on integrated voltages without requiring the computation of conventional mathematical slopes and all the extra circuitry necessary.

Similarly, Ghader et al. [19] validated that BP-free SNNs trained via FF can achieve competitive accuracy on benchmarks like MNIST, bolstering the proof that global error signals are not a requirement for high-performance spiking systems. With the verification of the plausibility of such an approach, our work innovates upon this foundation into our MIF model implementation, designing the layer-wise updates to depend on the local membrane potentials.

3.4 Memristive Integrate-and-Fire Neuron Dynamics

The spiking neuron adopted in our Memristive Spiking Neural Network (MSNN) is a memristive integrate-and-fire (MIF) unit, as depicted in Fig. 1. This neuron is implemented using a commercially available Knowm memristor model, allowing circuit-level simulations that closely reflect realistic device behavior. When a positive input current i is applied, the membrane capacitor accumulates charge and the membrane voltage v^t gradually deviates from the resting potential E_{rest} . As the voltage rises, the electric field across the reset memristor M_2 reaches its switching regime, temporarily coupling the membrane node to the reset potential E_{reset} . This induces a rapid voltage drop followed by a recovery process that returns the membrane voltage to E_{rest} , thereby generating an analog spiking waveform. Experimental demonstrations of this MIF spiking mechanism using physical Knowm devices have been reported in [20].

At the circuit-array level, MIF neurons are organized in a memristive crossbar structure, as illustrated in Fig. 2. In this configuration, a set of input currents $\{I_{\text{in}0}, \dots, I_{\text{in}4}\}$ is applied along the word lines of a 3×2 memristive crossbar, whose column currents drive a population of five MIF neurons and produce three output currents. The resting and reset voltages, E_{rest} and E_{reset} , are implemented as static voltage sources shared across neurons. Importantly, the effective membrane capacitance of each neuron is realized by the bit-line parasitic capacitance C_{BL} , originating from interconnect and transistor parasitics, allowing membrane integration to emerge naturally from circuit-level physical effects rather than from explicitly inserted capacitors.

The device-level dynamics are described using the Knowm memristor equations, which originate from the generalized metastable switch (MSS) modeling framework. In this formulation, the memristor conductance G and its series-limited counterpart G_1 define the relationship between the applied device voltage v_M and the resulting current response. An internal state variable $x_{\text{MR}} \in [0, 1]$ characterizes the effective

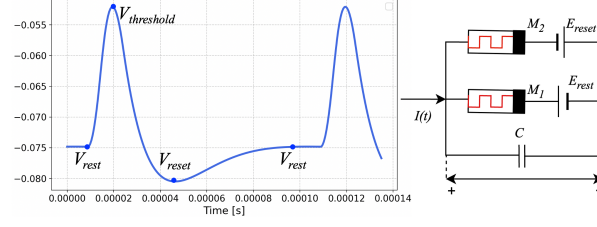


Figure 1: Schematic of the memristive integrate-and-fire (MIF) neuron. A membrane capacitor integrates the input current until the switching condition of the reset memristor is reached, leading to discharge and recovery to the resting potential, thereby generating an analog spike waveform. Reproduced with permission from X. Shi et al. 2025

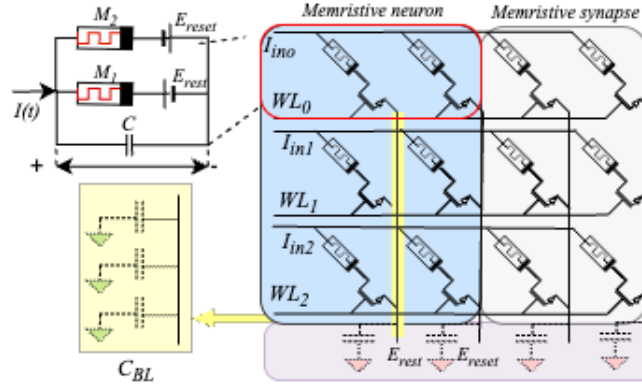


Figure 2: Circuit schematic of a fully memristive neural network. C_{BL} represents the parasitic bit-line capacitance. Reproduced with permission from X. Shi et al. 2025

switching state of the device:

$$\begin{cases} i = Gv_M = G \frac{1}{1+GR} v = G_1 v, \\ \dot{x}_{MR} = \frac{1}{\tau} \left(1 + e^{-\beta \left(\frac{1}{1+RG} v - V_{ON} \right)} \right) (1 - x_{MR}) \\ \quad - \frac{1}{\tau} \left(1 - \left(1 + e^{-\beta \left(\frac{1}{1+RG} v + V_{OFF} \right)} \right)^{-1} \right) x_{MR}, \end{cases} \quad (1)$$

where R_{ON} , R_{OFF} , V_{ON} , V_{OFF} , and τ denote device-specific parameters, while $\beta = 1/V_T$ represents the inverse thermal voltage.

To enable time-discrete forward simulation of the MIF neuron dynamics, the continuous-time equations are mapped to a discrete-time form using the forward Euler integration scheme. Compared with implicit solvers or higher-order numerical methods such as backward Euler or Runge-Kutta approaches, forward Euler provides an explicit and computationally lightweight update that supports efficient step-wise forward propagation over time. The membrane voltage evolves according to [10]:

$$v^{t+1} = \frac{I^t - G_1^t (v^t - E_{rest}) - G_2^t (v^t - E_{reset})}{C} + v^t, \quad (2)$$

where I^t denotes the synaptic input current, G_1^t and G_2^t correspond to voltage-dependent conductances associated with leakage and reset pathways, and C is the effective membrane capacitance, which in the crossbar implementation is dominated by the bit-line parasitic capacitance C_{BL} .

The conductance modulation of the neuron is governed by internal state variables x_1 and x_2 , each associated with a memristive element in a differential pair. These variables evolve as nonlinear functions of the membrane voltage and determine the instantaneous conductance levels. As an example, the update rule for x_1 is expressed as [10]:

$$x_1^{t+1} = \frac{1}{\tau_1} \left(\frac{1-x_1^t}{1+\exp\left(\frac{v_{on1}-(v^t-E_{rest})}{V_{th}k_v}\right)} - \frac{x_1^t}{1+\exp\left(\frac{(v^t-E_{rest})-v_{off1}}{V_{th}k_v}\right)} \right) + x_1^t. \quad (3)$$

A corresponding update is applied to x_2 , yielding complementary conductance dynamics that collectively realize stable integrate-switch-reset behavior without relying on explicit digital thresholding circuitry.

To avoid idealized impulse-based synaptic inputs, presynaptic activity is modeled using alpha-shaped current responses driven by spike trains [10]:

$$\tau_{syn} \frac{dI}{dt} = a - I, \quad \tau_{syn} \frac{da}{dt} = -a + W_i \sum_n \delta(t - t_i^n), \quad (4)$$

where a is an auxiliary current variable and W_i denotes the synaptic weight. This representation is well suited to analog circuit implementations and captures graded temporal dynamics commonly observed in biological and neuromorphic systems [21].

All neuronal and memristive state variables are evolved using explicit forward-time updates. This formulation maintains the physical fidelity of the MIF neuron and supports efficient integration with forward-only learning schemes in memristive neuromorphic architectures.

3.5 Forward-Forward Learning with Temporal Memristive Dynamics

The overall training procedure is summarized in Algorithm 1, which details the layer-wise forward execution, contrastive goodness evaluation, and greedy parameter updates implemented on memristive crossbar layers. The network is trained using the Forward-Forward (FF) algorithm, which replaces global back-propagation with a greedy, layer-wise learning rule based entirely on local activity measurements. Unlike conventional FF implementations that operate on static activations or firing rates, the proposed approach is explicitly temporal and is tightly coupled to the physical dynamics of the Memristive Integrate-and-Fire (MIF) neuron.

For each training sample, learning is driven by two forward simulations: a positive pass and a negative pass. This positive-negative pairing and label overlay process corresponds to Lines 3–6 in Algorithm 1. In the positive pass, the input image is augmented with its correct label using a spatial one-hot encoding. In the negative pass, the same image is paired with an incorrect label. Both versions are converted into temporal input currents using the synaptic alpha filter and are simulated for a fixed duration of T discrete time steps using the full MIF dynamics. The resulting temporal inputs are propagated through the network in a strictly layer-wise manner, as shown in Lines 8–15 of Algorithm 1, where each layer performs forward simulation,

Algorithm 1 Forward-Forward Training

Require: TrainLoader, $Scale_1$, L (Number of Layers)

```
1: for epoch = 1 to 20 do
2:   for each (Images, Labels) in TrainLoader do
3:     // Data Augmentation with Labels
4:      $\mathbf{x}_{pos} \leftarrow \text{OverlayCorrect}(\text{Images}, \text{Labels})$ 
5:      $\mathbf{x}_{neg} \leftarrow \text{OverlayIncorrect}(\text{Images}, \text{Labels})$ 
6:     // Initial Filtering and Scaling
7:      $\mathbf{h}_{pos}, \mathbf{h}_{neg} \leftarrow \text{AlphaFilter}(\mathbf{x}_{pos}, \mathbf{x}_{neg})$ 
8:      $\mathbf{h}_{pos} \leftarrow \mathbf{h}_{pos} / Scale_1, \quad \mathbf{h}_{neg} \leftarrow \mathbf{h}_{neg} / Scale_1$ 
9:     // Layer-wise Forward Pass
10:    for  $i = 0$  to  $L - 1$  do
11:       $\mathcal{L}_i \leftarrow \text{Layers}[i]$ 
12:       $\mathbf{v}_{pos}, \mathbf{v}_{neg}, \text{loss} \leftarrow \mathcal{L}_i.\text{forward}(\mathbf{h}_{pos}, \mathbf{h}_{neg})$ 
13:       $\mathbf{h}_{pos} \leftarrow \mathbf{v}_{pos} / Scale_1, \quad \mathbf{h}_{neg} \leftarrow \mathbf{v}_{neg} / Scale_1$ 
14:       $\mathcal{L}_i.\text{reset}()$ 
15:    end for
16:    // Final Layer Update
17:     $\text{loss} \leftarrow \text{LastLayer.train}(\mathbf{h}_{pos}, \mathbf{h}_{neg})$ 
18:    if batch % frequency == 0 then
19:      EVALUATE_ACCURACY(TestSet)
20:    end if
21:  end for
22: end for
```

local goodness evaluation, and state reset independently.

For a given layer, a local goodness metric is computed from the temporal evolution of the membrane voltage. Goodness is defined as the time-averaged mean squared membrane potential across all neurons in the layer,

$$g = \frac{1}{T} \sum_{t=1}^T \left(\frac{1}{N} \sum_{i=1}^N v_{i,t}^2 \right), \quad (5)$$

where $v_{i,t}$ denotes the membrane voltage of neuron i at time step t . By operating directly on membrane voltage rather than discrete spikes or firing rates, this objective aligns the FF criterion with the analog state variables of the memristive substrate and naturally incorporates temporal integration and device memory.

Learning is performed using a contrastive objective that encourages higher goodness for positive samples than for negative samples. A soft-hinge loss implemented via a softplus function is used,

$$\mathcal{L} = \ln(1 + \exp(\theta - (g_{\text{pos}} - g_{\text{neg}}))), \quad (6)$$

where θ is a fixed margin. This loss is minimized locally for each layer without propagating error signals to other layers.

Only the synaptic weights mapped to memristive conductances are optimized during training. The internal memristor state variables (x_1, x_2) evolve solely according to their voltage-dependent physical dynamics and are not directly updated by the learning rule. This preserves hardware fidelity while allowing learning

to emerge from voltage-driven activity.

Training follows a strictly greedy protocol. When optimizing layer l , all preceding layers are frozen and act as fixed temporal feature extractors, while deeper layers remain inactive. The temporal voltage output of each layer is L_2 -normalized before being passed to the next layer. This normalization prevents trivial solutions in which downstream layers maximize goodness by amplifying upstream activity and encourages the extraction of progressively higher-level representations.

4 Implementation

4.1 Technical Details

The code is originally developed using Python 3.13 and the PyTorch2 (CUDA 13.0) ecosystem. Local development utilized an NVIDIA GTX 1650 Ti mobile GPU paired with an AMD Ryzen 7 4800H iCPU and 16GB of 3200 MT/s SODIMM RAM. To accelerate the performance of the code, the system was migrated to superior hardware as well as the use of a multitude of software optimizations. The code was successfully ran on Google Colab and Kaggle, both using NVIDIA T4 accelerators and Python 3.11. The code was constructed to facilitate automatic saving of checkpoints with systematic naming, and basic python logging provides debug information to be used for future post-training analysis. When saving, the entire network is saved to ensure inter-layer compatibility across the network.

4.2 Optimizations

In order to maximize software optimizations, several hardware and software optimizations were implemented. The Triton library was utilized via `torch.compile` to fuse mathematical kernels within the MIF temporal step. The `fullgraph=True` constraint was applied, allowing the entire logic to be compiled into a pure, Python-free computational graph. The code incorporates the accommodation for GPU hardware compatibility, employing an automated check for CUDA-enabled GPUs, offloading any tensor computations to the device while maintaining CPU fallback support. To further streamline the data loading process, memory pinning and non-blocking transfers were enabled, allowing the CPU to prepare batches asynchronously while the GPU is actively processing current data.

Besides, the software features an extensive use of in-place torch operations and tensor pre-allocations to reduce any memory allocation costs for its mathematical calculations. Also, torch optimization tweaks were enabled to further accelerate training, such as the permitting of TensorFloat-32 (TF32), cuDNN auto-tuner via benchmark mode and 'high' matmul precision, dramatically speeding up performance in modern NVIDIA architectures. During the training loop, The Adam optimizer was utilized, paired with a condensed 100 total steps to minimize training times, without sacrificing accuracy.

4.3 MIF module

To create the Memristive Integrate-and-Fire (MIF) module, we apply a the object-oriented features of the PyTorch framework. For this module, we extended the `nn.Linear` base class, allowing it to inherit weight

initialization, parameter registrations and standard functionalities like `.backward()`. Within the `init` method we define all the memristor parameters such as R_{on} , R_{off} , and capacitance (C). To minimize computational overhead, we pre-allocate all the required tensors here, reducing the memory pressure on the GPU. Lastly, We also compile the `mem_step` function, which contains the core equations, with `torch.compile` and `fullgraph=True` to transform Python logic into a fused CUDA kernel, significantly increasing execution speed.

The core of the MIF lies in the `mem_step` function. This function contains the discrete system necessary to update the physical states of the simulated memristor at each time step t . One of the main issues of the MIF neuron is its catastrophic tendency to cause an activation explosion. To mitigate this, the membrane potential v is updated using a semi-implicit Euler scheme. By solving for v_{t+1} algebraically, it becomes self-normalizing:

$$v_{t+1} = \frac{v_t + \frac{I_{in} + G_1 E_1 + G_2 E_2}{C}}{1 + \frac{G_1 + G_2}{C}} \quad (7)$$

Where I_{in} is the resulting current from the linear transformation $Wx + b$. Unlike an explicit update, the system remains stable regardless of the magnitude of the input current or the conductances (G_1, G_2), preventing NaN errors through the accumulation of errors. To model the internal state variables x_1 and x_2 which are responsible for the low and high resistance states of the memristor, we use the threshold-dependent sigmoidal activation:

$$x_{i,t+1} = x_{i,t} + \frac{1}{\tau} \left[(1 - x_{i,t}) \cdot \sigma \left(\frac{(v - E_i) - v_{on}}{k_{th}} \right) - x_{i,t} \cdot \sigma \left(\frac{v_{off} - (v - E_i)}{k_{th}} \right) \right] \quad (8)$$

The final stage is the calculation of the physical conductance values G_1 and G_2 . This is implemented using a linear interpolation between the device's minimum and maximum conductance states, as shown respectively:

$$G_i = \frac{x_i}{R_{on}} + \frac{1 - x_i}{R_{off}} \quad (9)$$

To ensure mathematical stability and integrity, we clamp high risk variables. v is clamped to $[0.0, 1000.0]$ to prevent voltage spikes, while x_1 and x_2 are clamped to $[0.0, 1.0]$ to ensure they remain within bounds.

The training logic is encapsulated in the `train_layer` and the `forward` method, which implements the Forward-Forward (FF) learning algorithm a specified in the equations 5 and 6. Because our computational graph is layer-wise greedy, we use the `.detach()` method on variables that are not necessary for the local backward learning. State variables are also detached between time steps to contain the computational graph to the local layer. Due to the complexity of the prediction, we encapsulated the `predict` function into the module, where it runs a single forward pass across all the different labels.

4.4 System Configuration and Layer Hierarchy

We constructed a modular MSNN architecture consisting of a 784-unit input stage followed by two successive 512-neuron MIF layers. Rather than a singular global optimization, the network operates as a stack of independent processing blocks optimized via local activity targets. Each computational pass is discretized into 100 temporal increments, providing a sufficient window for the memristive internal states to evolve

and integrate incoming signals.

The transition from static image features to the temporal domain is handled by a synaptic alpha-filter module. This component transforms the 784 pixel intensities into a decaying current profile (I) with a time constant of $\tau_{\alpha} = 4.0$. This current serves as the primary stimulus for the initial MIF layer, where synaptic weights are mapped to memristive conductances that modulate the charge accumulation across the modeled neural membranes.

To preserve the information-theoretic value of deeper layers, we implement a rigorous normalization protocol between modules. Before the temporal output of one layer reaches the next, it is subjected to L_2 vector normalization and scaled down by specific constants ($scale_1 = 128$, $scale_2 = 8192$). This stabilization mechanism prevents signal saturation and ensures that the secondary and tertiary layers are forced to extract orthogonal, higher-level features rather than mimicking the activity patterns of the preceding stage.

4.5 Dataset

For ease of comparison, our model is evaluated on the datasets MNIST, FMNIST and KMNIST. These datasets, each with 60,000 training images and 10,000 testing images of 28×28 handwritten digits are utilized to train and evaluate the performance of our model. For a fair comparison - for each dataset, a model will evaluate all 10,000 testing images to achieve its results and will be ran with the same exact parameters.

Samples from a dataset are compiled into batches and transformed into an intensity based spiking source. To simulate biological synaptic decay, an alpha filter is applied to the input stream. The input's pixel intensity is transformed into magnitude peaks through the filter and pulses for 5 times ($T=100$, $step=20$).

$$\tau_{\alpha} = 4 \quad (10)$$

$$a[t + 1] = input[t] + \frac{-a[t]}{\tau_{\alpha}} \quad (11)$$

$$I[t + 1] = I[t] + \frac{a[t] - I[t]}{\tau_{\alpha}} \quad (12)$$

Due to the lack of global backpropagation, labels are spatially embedded into the images. To do so, the first 10 pixels of each image are reserved for a one-hot representation of the class label. For a digit n , the n th pixel is set to the maximum intensity value, while the remaining reserved pixels are suppressed to zero. This process generates the positive sample for training.

It is critical for the Forward-forward (FF) algorithm to generate good negative samples. The negative image is generated by taking the same positive image in the positive pass but is given an incorrect label. This approach appears to be sufficient for the network to associate the label and its intended classification.

4.6 Training and Inference

To train the first MIF layer, the historical record of the alpha filter's output is passed as the primary temporal input to the neurons. A local loss is calculated based on the differential goodness between positive and negative samples across the entire simulation time-window. The weights are updated according to this



Figure 3: This figure shows an example of the positive negative input pair. The positive sample (Left) has a correctly placed white pixel on the 0th position and the negative sample (Right) has a incorrectly placed white pixel on the 6th position

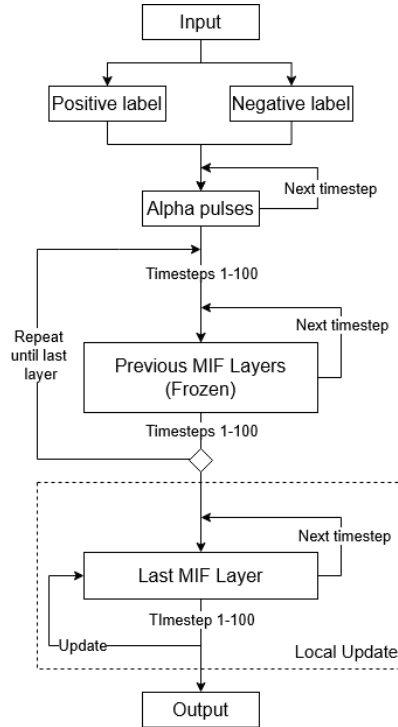


Figure 4: Diagram showing a training pass

local loss via the Adam optimizer, and the process is repeated for each sample batch throughout an epoch. Saved layers can also be loaded as a continuation from previous training epochs.

To train subsequent layers, all preceding layers are frozen in a static state, effectively acting as fixed feature extractors. The input data is passed through these normalized, static layers to generate the specific activation patterns required for the current active training layer. This layer-wise normalization is critical; by zero-centering and scaling the activity, it prevents downstream layers from redundantly learning low-level features already captured by earlier stages, forcing the deeper layers to extract higher-order information.

During the inference phase, the network utilizes a competitive goodness evaluation to determine the final classification. A single test image is duplicated ten times, and each copy is assigned a unique candidate label from 0–9. These labeled instances are processed by the network, which calculates the “goodness” at every layer. Unlike standard networks that rely on a final softmax layer, the classification decision here is governed by the highest cumulative goodness across the entire hierarchy.

Mathematically, the cumulative goodness $\mathcal{G}_{total}$ for a candidate label y is the summation of the goodness scores from all L layers:

$$\mathcal{G}_{total}(x, y) = \sum_{i=1}^L G_i(x, y) \quad (13)$$

where $G_i(x, y)$ represents the mean squared voltage accumulation in layer i . The final prediction $\hat{y}$ is selected as the label that maximizes this total activity:

$$\hat{y} = \arg \max_y \mathcal{G}_{total}(x, y) \quad (14)$$

By summing the goodness across all layers rather than just the final one, the network ensures that both low-level structural features and high-level abstract representations contribute to the robustness of the final digit recognition.

5 Testing

5.1 Training Loss and Data Integrity

To ensure the success of training process, the software architecture utilizes a dedicated extraction program to parse loss information from persistent log files. This approach ensures that every individual step of the training data is preserved even in the event of hardware interruptions or runtime exceptions. With the extraction of the training data, the loss history as featured in Figure 5 can be reconstructed at any interval, offering robust data loss mitigation. The checkpointing system is a crucial backup system for loss mitigation, allowing loss minimal restoration at any interval. Unlike models using backpropagation, each layer in our model possesses an independent loss, thus a visual confirmation of the difference in layer-wise loss curves also serves as a potent validation as empirical validation that the layers are operating as a "greedy" learner.

5.2 MIF Single Neuron Simulation

A simulation was conducted to record the outputs of a single MIF neuron over 100 steps, portraying the evolution of its alpha filter and memristive variables. Fig. 6 illustrates the current I from the alpha filter, which exhibits a biologically inspired natural decay. Fig. 7 shows the resulting evolution of the membrane potential V in response to this current. The internal state variables x_1 and x_2 express distinct, smooth temporal decays, with x_2 decaying more rapidly than x_1 . The observed interplay between the decaying alpha current and the differential evolution of x_1 and x_2 demonstrates that the MIF neuron successfully captures the multi-scale temporal dynamics necessary to represent complex data patterns within its physical state variables.

5.3 Explosion Test

To validate the necessity of the use of our semi-implicit Euler integration, an isolated comparative stress test was conducted between the 2 distinct Euler methods. Both methods were subjected to a "Worst-Case

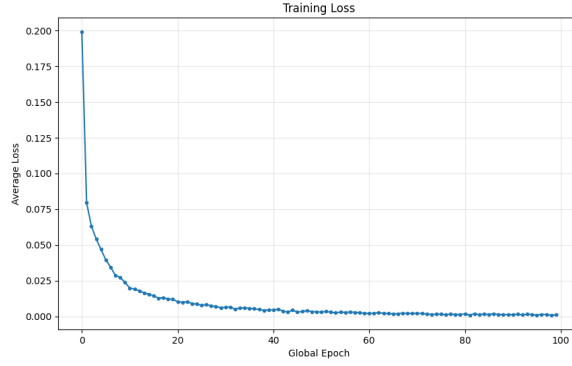


Figure 5: Diagram of reconstructed training loss from log files.

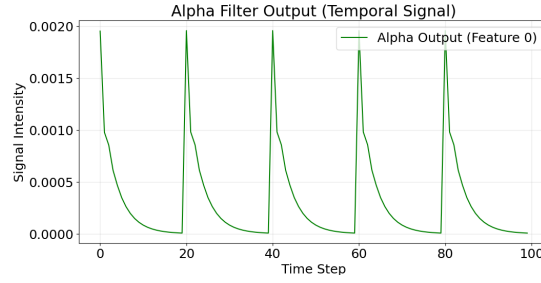


Figure 6: Alpha historical record output. The green line represents the the current "Scenario" input, consisting of a constant input of all ones for 100 time steps. Since the test's objective is to only compare the stability of the 2 methods, internal clamping was disabled. If a NaN (Not-a-Number) value was observed in any of the methods, it would indicate a floating-point overflow, indicating that the method was unsuitable for training.

Table 1: Implicit vs. Explicit Euler Integration

Metric	Explicit Euler (Prone)	Semi-Implicit Euler (Proof)
Input Signal (I_{in})	1.0 (Constant)	1.0 (Constant)
Temporal Steps (t)	100	100
v_{pos} NaN Detected	True	False
g_{pos} NaN Detected	True	False
Outcome	FAILED	PASSED

For the explicit implementation, the membrane potential explodes due to the cascading error of x_1 and x_2 , pushing the value into NaN. In comparison, The semi-implicit implementation proves itself able to withstand a "Worst-Case Scenario" stress test, confirming that its algebraic logic is indeed self-dampening and ensuring its ability to handle any dataset.

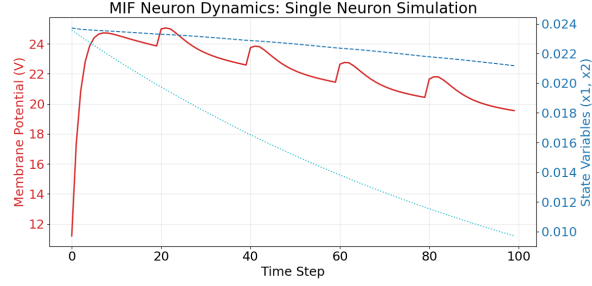


Figure 7: MIF historical record output. The red line represents V , the blue dashed line represents x_1 and the cyan dotted line represents x_2

6 Experiment

6.1 Result

Our proposed model was evaluated on the MNIST, FMNIST and KMNIST datasets using identical hyperparameters optimized for MNIST to assess its performance on tasks of varying complexity. All models were trained for 20 epochs on a batch size of 64, threshold of 1, learning rate of 0.001 and utilized the full 10k images in the test sets.

On MNIST the network achieved 97.47% using a single 512 MIF layer, with a slight improvement to 97.58% with the addition of the second layer. This indicates that a single memristive layer captures most of the discriminative features of the simpler MNIST dataset. For FMNIST and KMNIST dataset, which features higher complexity, the second layer led to substantial gains, improving accuracy from 86.23% to 87.95% and from 87.83% to 89.23%, respectively. These results indicate that deeper memristive layers are likely beneficial for more complex patterns. Subsequent tests on MNIST showed a peak accuracy of 98.21% with an extended duration of 40 epochs (omitted from Table 2 for cross-dataset consistency).

It should also be noted that the output from the first layer shown in Fig.7 has a significant difference from the output of the alpha filter shown in Fig.6, which might impact the performance of the model. All results and comparisons are shown in Table 2.

6.2 Accuracy Comparison

The performance of our proposed work was benchmarked against several state-of-the-art implementations of the Forward-Forward (FF) algorithm and backpropagation (BP), as summarized in Table 2. Our two-layer architecture (512-512) achieved a peak accuracy of **97.58%** on the MNIST dataset, directly matching against the performance of complex software-based SNNs like those proposed by Ororbia et al. [17]. Notably, even with a single-layer configuration (512 hidden neurons), the model maintained a high classification accuracy of 97.47% on MNIST and 86.23% on Fashion-MNIST, demonstrating significant robustness despite the architectural simplification.

When compared to existing hardware-centric models, our system significantly outperforms the memristive SNN by Prajapat et al. [8], which utilizes traditional backpropagation and achieves only 91.27%. This performance gap suggests that the Forward-Forward learning paradigm is exceptionally well-suited for mem-

Table 2: Comparison of Accuracy and Architecture: Proposed Work vs. SOTA Works

Model	Architecture	Epochs	Type	Learning	Neuron Feature	MNIST	FMNIST	KMNIST
[9]	2k-2k-2k-2k	60	Software ANN	FF	Standard ANN	98.84	-	-
[14]	500-500-500	150	Software ANN	FF	Standard ANN	97.9	88.4	-
[15]	500-500-500	100	Software ANN	FF	Standard ANN	98.68	89.67	-
[17]	5k-2k	30	Software SNN	FF	IF / LIF	97.58	-	91.53
[19]	500-500	300	Software SNN	FF	LIF	98.34	89.36	91.26
[18]	40-120-120-240	30,50	Software SNN	FF	LIF	98.17	86.32	-
[16]	1000-10	140	Software SNN	BP	LIF	97.91	89.05	-
[8]	397-204-10	7	Memristive SNN	BP	Digital	91.27	-	-
[22]	1Mem-C-M-C-M-1024F-10F	-	Memristive SNN	BP	Mixed-signal	97.10	-	-
[23]	C-M-C-M-10F	300	Memristive SNN	BP	Digital	~96.00	-	-
[24]	100-10	100	Memristive SNN	BP	Analog	83.24	-	-
[25]	100-10	50	Memristive SNN	BP	Analog	93.18	84.97	-
Our work	512	20	Memristive SNN	FF	Analog	97.47	86.23	87.83
Our work	512-512	20	Memristive SNN	FF	Analog	97.58	87.95	89.23

Table 3: Architecture Settings

Model	Settings
MIF Physics	$R_{on} = 1000; R_{off} = 100000; v_{on} = 110; v_{off} = 5;$ $\tau = 100; E_1 = 0; E_2 = 50; C = 100 \times 10^{-6};$ $k_{th} = 15; v_{init} = 0.0238; T = 100; step = 20$
FF Training	batch_size=64; $\theta = 1.0; lr_1 = 0.001; lr_1 = 0.01; epochs=20;$ $\tau_\alpha = 4; labels=one-hot (pixels 0-9)$

ristive substrates, as it bypasses the precision requirements and global weight-update challenges inherent in BP. Furthermore, while software-only ANNs ([9], [15]) maintain a slight edge in absolute accuracy, our proposed work achieves comparable results while operating within the physical constraints of memristive state variables (x_1, x_2). Although not shown on the table, the extended 40 epoch version of the MNIST trained model achieved a higher accuracy of **98.21%** than the similar sized FF ANN implementation [14]. This suggests that the biological plausibility of the MIF neuron does not come at the cost of computational utility, bridging the gap between high-performance machine learning and energy-efficient neuromorphic hardware.

6.3 Energy and Area Efficiency

The proposed system operates entirely in the analog domain, avoiding the need for repeated analog-to-digital conversion and the associated power and area overhead. As a result, the architecture offers a substantially more compact and energy-efficient realization compared to digital or mixed-signal neural accelerators.

For a network configuration of [784, 512, 512], the model comprises approximately 663.5k synaptic con-

nections, realized using 1.33M RRAM devices in a differential configuration. Based on area estimates reported for a 65 nm CMOS technology node [26], the resulting memristive crossbar arrays occupy approximately 0.23 mm^2 . In contrast, an equivalent digital ANN implementation with 8-bit precision would require multiple SAR ADCs per crossbar tile. For the same network size, the corresponding ADC-dominated design would exceed 1.2 mm^2 , yielding an area overhead of roughly $5.2\times$.

Power consumption is estimated under conservative operating assumptions. We assume an average neuron activation rate of 2%, a mean memristor resistance of $R_{\text{avg}} = 50.5 \text{ k}\Omega$, and an average operating voltage of $v_{\text{avg}} = 57.5 \text{ mV}$. The total average power is then approximated as

$$P_{\text{total}} = \frac{v_{\text{avg}}^2}{R_{\text{avg}}} \times N_{\text{cells}} \times 2\% \approx 1.74 \text{ mW}, \quad (15)$$

where N_{cells} denotes the total number of active memristive devices. This estimate indicates a substantially lower power footprint compared with digital counterparts. For reference, the ADCs alone required in a digital-equivalent architecture would consume approximately 8.21 mW, exceeding the total power budget of the proposed analog system.

Inference latency is likewise reduced by exploiting the intrinsic RC dynamics of the memristive substrate. Unlike ADC-based designs, where conversion latency can exceed 7 ms for 1,000 simulation steps, the proposed architecture relies on analog time constants for state evolution. Assuming an average delay of 100 ns per simulation step, a complete inference pass—consisting of ten label evaluations with 100 steps each—can be completed within approximately $100 \mu\text{s}$. Under these conditions, the corresponding energy cost is estimated at 174 nJ per input image, highlighting the suitability of the proposed memristive system for low-latency, energy-constrained edge computing applications.

7 Conclusion

7.1 Evaluation

Experimental evaluations demonstrate that the proposed framework achieves competitive performance on standard vision benchmarks, reaching 97.58% accuracy on MNIST. Importantly, this performance is obtained using a pure FF learning scheme without surrogate gradients, auxiliary optimization tricks, or hybrid training strategies. These results suggest a strong compatibility between local goodness-based learning and the intrinsic voltage-driven dynamics of memristive integrate-and-fire neurons, indicating that effective learning can emerge directly from the underlying device physics rather than from algorithmic complexity. Our energy usage calculations estimate memristors to be 4.7x more efficient than ADCs alone. If we include the power for digital logic, memory access (SRAM), and the GPU/NPU cores, our model is likely 10x to 50x more power-efficient than a standard mobile-grade AI chip.

In totality, this work presents a fully memristive neuromorphic architecture that integrates Memristive Integrate-and-Fire (MIF) neurons with the Forward-Forward (FF) learning paradigm. By adopting a strictly layer-wise, forward-only training protocol, we successfully establish that our proposed approach eliminates the need for global error backpropagation and its associated memory-intensive computational graphs. As

a result, learning is achieved using only locally available activity measurements that is a natural fit with the physical dynamics of memristive devices. While this work considers relatively shallow architectures and fixed training schedules, it establishes a solid foundation for scalable compute-in-memory learning. Overall, the proposed framework points toward practical, energy-efficient neuromorphic learning on fully analog edge hardware.

7.2 Future Work

Future research will focus on scaling the proposed model by the addition of more hidden layers to investigate if higher-level feature extraction can be achieved to improve overall accuracy. As stated by Lorberbom [14], FF models tend to struggle with information retention across deep architectures, thus more work needs to be done to verify and optimize our proposed model. Another potentially crucial improvement is research into the faster inference times by integrating a traditional classification as the output layer. This might be near mandatory for datasets that have a large number of classes. Additionally, evaluating the model on temporal datasets, namely the DVS dataset, will be an important step for testing the model's ability to encode time-dependent information.

While the FF algorithm removes the need for backpropagation, the "NoProp" network, takes it a step further. The forward propagation of activity is replaced by local, decentralized updates, potentially offering even greater biological plausibility and parallelization within memristive hardware. Our preliminary findings indicate that loss function calculation is the main contributor of model accuracy, as such a natural progression is the research into more advanced reward mechanisms like hormone modulation networks, where global "neuromodulators" can adjust the learnability of individual neurons based on overall performance. Since the FF learning rule relies on the generation of negative data, an imaginative "sleeping" period like in Google's Dreamerv3 could be of use to hallucinate sophisticated, adversarial negative samples for more effective for training performance.

Architecturally, the absence of backpropagation in FF allows for the exploration of novel approaches. An example would be ability to engineer an architecture to iterate internally until a specific confidence threshold is met, similar to the behavior of modern reasoning models. To support this, research on memory-retention mechanisms will be necessary to mitigate catastrophic forgetting. Lastly, when memristor hardware becomes more accessible, we intend to benchmark the model on energy efficiency, spatial requirements, and hardware-level performance metrics.

7.3 Limitations

As with all models, there exist limitations to the utility and generalizability of our findings. Our current implementation represents the signal strength through voltage amplitude and a simulated alpha pulse. Due to lack of depth and simplified nature of our model, the pulses across the network are largely synchronized. While effective for this study, biological systems often rely on asynchronous spike-timing, suggesting our model may not have yet achieved optimal energy or computational efficiency. Currently, while our model serves as a valuable proof of concept, it does not currently outperform state-of-the-art Convolutional Neu-

ral Networks (CNNs) in raw accuracy. We also encountered challenges in replicating the performance gains from peer inhibition in Hinton's original Forward-Forward work [9], suggesting that our memristive implementation may be introducing unexplored complex dynamics.

From a hardware perspective, this model has been evaluated entirely via software simulation, thus real-world performance on physical memristive hardware may vary greatly due to non-ideality, thermal noise, and manufacturer implementations. Besides, the model remains largely a "black box", offering limited utility for Explainable AI (XAI). This lack of interpretability is moreso an architectural trait rather than a learning one, as such, it is possible entirely possible to implement our work in an explainable manner. Finally, while the Fully Memristive Spiking Neural Network serves as a robust alternative to traditional architectures, it remains an abstraction of biological reality. The sheer degree of variance and stochasticity found in biological neurons is a feature currently missing from most neural network implementations, which may be a critical factor in achieving truly brain-like plasticity and adaptability.

7.4 Implications

On a broader scale, as development into highly efficient, low-power models continues to advance, some ethical and societal concerns might arise. While still in its infancy, future discussion on the governance of edge-AI deployment in autonomous weaponry or surveillance systems will soon become a necessity. Moreover, if low powered low-power generative agents becomes commonplace, the cost of running a massive pod of AI entities become even more feasible, inevitably contributing to the "Dead Internet Theory" on an unprecedented scale, flooding human interaction platforms with AI-generated content. Such a phenomenon will cause the internet to become an unreliable source for human interaction and transform the internet into a potent tool for psychological operations, introducing novel social challenges for the modern generation. Additionally, as AI approaches human-level reasoning, it may alter fundamental human relationship dynamics and prompt deeper philosophical debates regarding AI rights and personhood.

However, the positive impacts of decentralized, low-power AI cannot be ignored. In remote or developing regions without stable power grids or high-speed internet, neuromorphic hardware offers economically viable solutions for critical applications such as medical diagnostics and agricultural sensing. By integrating intelligent edge-AI into devices like smartwatches, hearing aids, and environmental drones, we can democratize access to advanced computation. With the improved efficiency of memristive hardware, we can greatly reduce the carbon footprint associated with large-scale model training. It must be warned that although the power usage of neuromorphic hardware is much lower, Jevons Paradox suggests that such efficiency gains could paradoxically lead to a massive scaling in the number of AI deployments, offsetting these energy savings at a global scale. Nonetheless, this would likely still be a net positive, providing smarter, more integrated tools for everything from industrial servers to everyday tasks.

Finally, there is the deeply relevant aspect of privacy. As countries and companies try to store as much of your data as possible, the lower power costs mean a truly intelligent personal assistant could run entirely on a personal device. This would allow them to be completely personalized to the user's workflow without sending private data to a cloud server. Ultimately, we believe this work is a step toward a future where AI is private, efficient, accessible and an indispensable tool for the progress of human flourishing.

References

- [1] H. Y. Ooi. (2026) MemristorNN. EECS GitLab Repository. [Online]. Available: <https://gitlab.eecs.qub.ac.uk/40374182/memristornn>
- [2] X. Shi, L. L. Minku, and X. Yao, "Evolving memristive reservoir," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 10, pp. 1–15, 2023.
- [3] X. Shi, Z. Zeng, L. Yang, and Y. Huang, "Memristor-based circuit design for neuron with homeostatic plasticity," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 5, pp. 359–370, 2018.
- [4] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.
- [5] L. O. Chua, "Memristor-the missing circuit element," *IEEE Transactions on Circuit Theory*, vol. 18, no. 5, pp. 507–519, 1971.
- [6] P. Wijesinghe, A. Ankit, A. Sengupta, and K. Roy, "An all-memristor deep spiking neural computing system: A step toward realizing the low-power stochastic brain," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 5, pp. 345–358, 2018.
- [7] J. Wang, S. Hu, X. Zhan, Q. Yu, Z. Liu, T. P. Chen, Y. Yin, S. Hosaka, and Y. Liu, "Handwritten-digit recognition by hybrid convolutional neural network based on hfo2 memristive spiking-neuron," *Scientific Reports*, vol. 8, no. 1, pp. 1–7, 2018.
- [8] S. Prajapat, M. N. Mondal, and S. Sur-Kolay, "Efficient and fault-tolerant memristive neural networks with in-situ training," *arXiv preprint arXiv:2507.20193*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.20193>
- [9] G. Hinton, "The forward-forward algorithm: Some preliminary investigations," *arXiv preprint arXiv:2212.13345*, 2022. [Online]. Available: <https://arxiv.org/abs/2212.13345>
- [10] X. Shi, P. Zhou, C. McTaggart, and X. Yao, "Memristive dynamical spiking neural networks with spatiotemporal heterogeneity," in *The 2025 International Conference on Machine Intelligence and Nature-inspired Computing (MIND)*. Institute of Electrical and Electronics Engineers Inc., 2025.
- [11] S. Grossberg, "Competitive learning: From interactive activation to adaptive resonance," *Cognitive Science*, vol. 11, no. 1, pp. 23–63, 1987.
- [12] Z. Wang, S. Joshi, S. Savel'ev, W. Song, R. Midya, Y. Li, M. Rao, P. Yan, S. Asapu, Y. Zhuo et al., "Fully memristive neural networks for pattern classification with unsupervised learning," *Nature Electronics*, vol. 1, no. 2, pp. 137–145, 2018.

- [13] P.-Y. Chen, X. Peng, and S. Yu, "Neurosim: A circuit-level macro model for benchmarking neuro-inspired architectures in online learning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 12, pp. 3067–3080, 2018.
- [14] G. Lorberbom, "Layer collaboration in the forward-forward algorithm," *arXiv preprint arXiv:2305.12393*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.12393>
- [15] J. Gong, R. Luo, E. Wang, L. Ge, B. Li, F. Marattukalam, and W. Abdulla, "Reshaping the forward-forward algorithm with a similarity-based objective," *arXiv preprint arXiv:2509.08697*, 2025. [Online]. Available: <https://arxiv.org/abs/2509.08697>
- [16] L. Taylor, A. King, and N. Harper, "Robust and accelerated single-spike spiking neural network training with applicability to challenging temporal tasks," *arXiv preprint arXiv:2205.15286*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.15286>
- [17] A. G. Ororbia, "Contrastive signal-dependent plasticity: Self-supervised learning in spiking neural circuits," *Science Advances*, vol. 10, no. 43, p. eadn6076, Oct 2024.
- [18] C. Xu, Z. Yang, Y. Liu, X. Liao, G. Mo, H. Zeng, and Y. Yang, "Ffgaf-snn: The forward-forward based gradient approximation free training framework for spiking neural networks," *arXiv preprint arXiv:2507.23643*, 2025. [Online]. Available: <https://arxiv.org/abs/2507.23643>
- [19] M. Ghader *et al.*, "Backpropagation-free spiking neural networks with the forward-forward algorithm," *arXiv preprint arXiv:2502.20411*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.20411>
- [20] S. M. Kang, D. Choi, J. K. Eshraghian, P. Zhou, J. Kim, B.-S. Kong, X. Zhu, A. S. Demirkol, A. Ascoli, R. Tetzlaff, and L. O. Chua, "How to build a memristive integrate-and-fire model for spiking neuronal signal generation," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 68, no. 12, pp. 4837–4850, 2021.
- [21] J. K. Eshraghian, S. Baek, J.-H. Kim, N. Iannella, K. Cho, Y. S. Goo, H. H. Lu, S.-M. Kang, and K. Eshraghian, "Formulation and implementation of nonlinear integral equations to model neural dynamics within the vertebrate retina," *International Journal of Neural Systems*, vol. 28, no. 07, p. 1850004, 2018.
- [22] J. J. Wang, S. G. Hu, X. T. Zhan, Q. Yu, Z. Liu, T. P. Chen, Y. Yin, S. Hosaka, and Y. Liu, "Handwritten-digit recognition by hybrid convolutional neural network based on hfo2 memristive spiking-neuron," *Scientific Reports*, vol. 8, no. 1, p. 12546, 2018. [Online]. Available: <https://doi.org/10.1038/s41598-018-30768-0>
- [23] P. Wijesinghe, A. Ankit, A. Sengupta, and K. Roy, "An all-memristor deep spiking neural computing system: A step toward realizing the low-power stochastic brain," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 5, pp. 345–358, 2018. [Online]. Available: <https://doi.org/10.1109/TETCI.2018.2829924>

- [24] Y. Yang, "Spiking neurons with spatiotemporal dynamics and gain modulation for monolithically integrated memristive neural networks," *Nature Communications*, vol. 11, p. 3399, 07 2020. [Online]. Available: <https://doi.org/10.1038/s41467-020-17215-3>
- [25] P. Zhou, D.-U. Choi, W. D. Lu, S.-M. Kang, and J. K. Eshraghian, "Gradient-based neuromorphic learning on dynamical rram arrays," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 12, no. 4, pp. 888–897, 2022.
- [26] Q. Wang, X. Wang, S. H. Lee, F.-H. Meng, and W. D. Lu, "A deep neural network accelerator based on tiled rram architecture," in *2019 IEEE International Electron Devices Meeting (IEDM)*. IEEE, 2019, pp. 14–4.

A Appendices

A.1 AI Acknowledgements

Google Gemini was used to proofread the draft, generate examples, and discover relevant academic sources. All ideas and intents are first written without AI assistance, and are then fed to the AI to suggest improvements to any incoherent flow, synonyms or reasoning errors. For code, Gemini usage is limited to debugging errors and generating tests, while the Windsurf extension for VSCode was used solely for code auto-completion. All resulting code and manuscripts are manually reviewed to ensure technical accuracy.